

Enhancing Space Imagery with DIP

A COURSE PROJECT

REPORT By

Konchada Shyam Sai - RA2311026010305

M Karthikeya - RA2311026010302

M S Hrithik - RA2311026010299

Under the guidance of

Dr. Megha Sahu

*In partial fulfillment for the
Course of*

21CSE251T - Digital Image Processing

In Computer Science & Engineering



FACULTY OF ENGINEERING AND

TECHNOLOGY SRM INSTITUTE OF

SCIENCE AND TECHNOLOGY

**Kattankulathur, Chengalpattu
District**

May 2025

Contribution Table :

Page Number	Topic	Contribution
3	Problem Definition	Karthikeya
4	Problem Explanation	Shyam Sai
5	Design Techniques	Hrithik
6-10	Algorithm	ShayamSai,Karthikeya
11-18	Implementation	Hrithik
19-28	Result	Karthikeya, Shyam
28	Conclusion	All

Problem Definition

Astronomy relies heavily on images captured by space- and ground-based telescopes to study celestial bodies and phenomena. However, these raw astronomical images often suffer from several limitations, such as low contrast, background noise, and blurring due to atmospheric interference or instrument imperfections. These issues can obscure important features like stars, galaxies, nebulae, or distant exoplanets. To analyze and interpret such data accurately, astronomers require enhanced, high-quality images. This is where Digital Image Processing (DIP) becomes a critical tool in the field of astronomical research.

Digital Image Processing techniques are used to improve the visual quality of astronomical images and to extract valuable information from them. Common tasks include contrast enhancement using histogram equalization or contrast stretching, noise reduction using smoothing filters, and detail enhancement through edge detection techniques such as Laplacian filters. Frequency domain processing, such as the use of the Discrete Fourier Transform (DFT), helps reveal periodic structures or patterns not easily visible in the spatial domain. These techniques make it possible to detect faint stars, distinguish overlapping objects, and correct image distortions caused by geometric transformations.

As astronomical data grows in volume and complexity, especially with the advancement of high-resolution telescopes like the James Webb Space Telescope (JWST), the need for effective image processing becomes even more crucial. Automated and interactive DIP techniques allow scientists to process large datasets efficiently, aiding in tasks such as detecting supernovae, classifying galaxies, and studying cosmic background radiation. By refining raw imagery, DIP enables more accurate observations and discoveries, playing an essential role in advancing our understanding of the universe.

Problem Explanation :

Astronomical imagery captured by telescopes such as the Hubble and James Webb Space Telescope plays a vital role in observing and studying distant celestial objects. However, raw images often come with significant challenges, including poor contrast, noise interference, and atmospheric distortions. These issues are primarily caused by the faint intensity of light from distant galaxies or stars, which is easily overshadowed by detector noise or cosmic rays. Additionally, factors like blurring from atmospheric turbulence or telescope misalignments further degrade image quality. These challenges limit the visibility of subtle features and make it difficult for scientists to conduct precise analysis. As a result, enhancing image quality through Digital Image Processing (DIP) becomes essential for extracting meaningful information from raw astronomical data.

To address these problems, a variety of DIP techniques are applied, each serving a unique function in image enhancement. Techniques like histogram equalization and contrast stretching are used to improve image contrast, making faint features more visible. Log and gamma transformations help emphasize dim regions while correcting brightness. Edge detection methods such as Laplacian filters are used to highlight object boundaries, which is crucial for identifying the structure of galaxies and planets. Geometric transformations correct for misalignments between images taken at different times or from different instruments, enabling accurate comparisons. Additionally, frequency domain analysis using Discrete Fourier Transform (DFT) helps identify repetitive patterns and suppress unwanted noise. These methods work collectively to refine image quality and make astronomical data more suitable for analysis.

The Python implementation of these DIP techniques makes it accessible for both students and researchers to experiment with astronomical imagery. Using libraries like OpenCV, NumPy, and Matplotlib, the code provides a modular and interactive framework for image processing. Users can load images, select specific transformations, and view side-by-side comparisons of the original and enhanced images. This hands-on approach allows for deeper understanding of each technique's effect. Whether it's correcting misalignments through geometric transforms or revealing hidden nebulae using log and gamma corrections, the tool demonstrates how powerful image processing can be when applied thoughtfully. The modular structure also allows for easy extension, making it a valuable foundation for further research and development in the field of astronomical image enhancement.

Design Techniques

Modular Function Design

- Each transform is a standalone Python function taking an RGB NumPy array and returning a processed array.
- Core steps (grayscale conversion, normalization, clipping) are repeated in many but kept internal to each function for clarity.

Separation of Concerns

- GUI and file I/O handled in main() via Tkinter and OpenCV.
- User interaction (choice of transform, parameter inputs) is centralized, while image-processing logic remains decoupled.

Manual Loop vs. Vectorization

- Most transforms use NumPy vector operations for efficiency (log, gamma, histogram equalization via lookup table).
- Laplacian edge detection is implemented via explicit loops for clarity, at the cost of speed.

Use of Standard Libraries

- OpenCV for image loading/conversion.
- Matplotlib for side-by-side visualization.
- Tkinter for cross-platform file dialog.

Algorithm for the problem

Input: A digital image file (e.g., input.jpg)

Output: A visually enhanced image

Point Transforms

- **Logarithmic:**
 1. Convert pixels to float.
 2. Compute scaling constant $c = 255 / \log(1 + \max)$.
 3. Apply $\text{out} = c * \log(1 + \text{img})$, clip to $[0, 255]$.
- **Gamma:**
 1. Float conversion.
 2. $c = 255 / (\max^\gamma)$.
 3. $\text{out} = c * \text{img}^\gamma$, clip.
- **Negative:**
 - $\text{out} = 255 - \text{img}$.

Spatial Transforms

- **Histogram Equalization:**
 1. Compute grayscale via dot product.
 2. Histogram and cumulative distribution (CDF).
 3. Normalize CDF to $[0, 255]$ and map each pixel.
 4. Stack back to three channels.
- **Contrast Stretching:**
 - Linear mapping from input range $[\text{in_low}, \text{in_high}]$ to output $[\text{out_low}, \text{out_high}]$.
- **Thresholding:**
 1. Convert to grayscale.

2. Binarize at threshold thresh: $\text{pixels} \geq \text{thresh} \rightarrow 255$, else 0.

Frequency Domain

- **DFT Spectrum:**

1. Grayscale conversion.
2. Compute 2D FFT and shift zero frequency to center.
3. Magnitude spectrum via $\log(1 + |F|)$.
4. Normalize to displayable range.

Geometric

- **Affine Warping:**

1. Build translation and rotation+scaling matrices.
2. Compute homogeneous coordinates of each pixel.
3. Map through transform, round to integer grid, and scatter original pixel values to new positions.

Edge Detection

- **Laplacian:**

- Convolve 3×3 Laplacian kernel via nested loops, take absolute, clip, and stack.

Formulas:

1.Log Transformation:

$$s = c \cdot \log(1 + r)$$

Where:

s = output pixel

r = input pixel

c = scaling constant

2. Gamma Correction (Power-Law Transformation):

$$s = c \cdot r^\gamma$$

Where:

γ = gamma value

c = scaling constant

3. Histogram Equalization:

$$s_k = (L - 1) \sum_{j=0}^k \frac{n_j}{MN}$$

Where:

s_k = output intensity

L = number of gray levels

n_j = number of pixels with intensity

M×N = image size

4. Contrast Stretching :

$$s = \frac{(r - r_{min}) \cdot (s_{max} - s_{min})}{r_{max} - r_{min}} + s_{min}$$

Where:

r = input intensity

s = output intensity

$r_{\min}, r_{\max}$ = minimum and maximum intensities in input

$s_{\min}, s_{\max}$ = desired output range

5. Discrete Fourier Transform (DFT):

$$F(u, v) = \sum_{x=0}^{M-1} \sum_{y=0}^{N-1} f(x, y) \cdot e^{-j2\pi(\frac{ux}{M} + \frac{vy}{N})}$$

Inverse DFT:

$$f(x, y) = \frac{1}{MN} \sum_{u=0}^{M-1} \sum_{v=0}^{N-1} F(u, v) \cdot e^{j2\pi(\frac{ux}{M} + \frac{vy}{N})}$$

6. Geometric Transformations:

Translation:

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} x \\ y \end{bmatrix} + \begin{bmatrix} \Delta x \\ \Delta y \end{bmatrix}$$

Rotation:

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} \cos \theta & -\sin \theta \\ \sin \theta & \cos \theta \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix}$$

Scaling:

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} s_x & 0 \\ 0 & s_y \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix}$$

7. Negative Transformation:

$$s = L - 1 - r$$

Where:

L = total number of gray levels

8. Thresholding:

$$s(x, y) = \begin{cases} 1, & \text{if } f(x, y) \geq T \\ 0, & \text{if } f(x, y) < T \end{cases}$$

9. Laplacian Edge Detection:

$$\nabla^2 f(x, y) = \frac{\partial^2 f}{\partial x^2} + \frac{\partial^2 f}{\partial y^2}$$

4 neighborhood Laplacian:

$$\begin{bmatrix} 0 & 1 & 0 \\ 1 & -4 & 1 \\ 0 & 1 & 0 \end{bmatrix}$$

Implementation

1. Language & Libraries

The project is developed using **Python 3.x**, leveraging several powerful libraries:

- **NumPy** is used for efficient numerical operations on images. It allows for fast array manipulations, crucial for image processing tasks like pixel transformations, histogram adjustments, and geometric modifications.
- **OpenCV (cv2)** is utilized for loading, converting, and manipulating images. It supports a variety of image formats (e.g., JPEG, PNG) and provides methods for reading, displaying, and processing images.
- **Matplotlib** is used for visualizing images and their transformations. It allows for easy display of the original and processed images side by side, enabling clear comparisons.
- **Tkinter** offers a simple, cross-platform GUI for file selection and parameter input, making the tool accessible for users who might not be familiar with command-line interfaces.

2. Structure

The code is organized in a modular way, with a central module `dip_module.py` containing the main image processing functions:

- Each transformation (logarithmic, gamma, histogram equalization, etc.) is encapsulated within a standalone function, allowing users to choose any transformation independently.
- These functions take the image (represented as an RGB NumPy array) as input, process it based on the selected transformation, and output the modified image. The core steps of grayscale conversion, normalization, and clipping are implemented

within each function, ensuring that the transformations are independent and reusable.

- The **main()** function serves as the central hub, handling user interactions, displaying the transformation menu, and showing the results.

3. User Flow

1. **File Selection:** When the program starts, a file dialog appears, allowing the user to select an image file. It supports various image formats, including common types like JPEG and PNG, and even FITS (used in astronomy).
2. **Displaying Information:** After loading the image, the program prints details such as the image's dimensions and data type, giving the user insight into the file being processed.
3. **Transformation Menu:** A list of possible transformations is displayed, with users selecting one (e.g., log transform, gamma correction) based on their needs.
4. **Parameter Input:** If the selected transformation requires additional parameters (such as gamma value for gamma correction or threshold value for binarization), the user is prompted to input these values.
5. **Result Display:** The original image and the transformed image are displayed side by side for comparison, helping the user visually assess the effect of the transformation.

4. Error Handling

- **File Handling:** The code checks if a file is selected. If no file is chosen or the file fails to load, the program exits with a message notifying the user of the error.
- **Input Validation:** For parameters that require numerical input (like gamma value or threshold), the program uses try/except blocks to handle invalid or non-numeric entries. This ensures that the program doesn't crash due to user input errors.

CODE :

```
import tkinter as tk
from tkinter import filedialog
import numpy as np
import matplotlib.pyplot as plt
import cv2

# ----- Manual DIP Function Implementations -----

def log_transform(img):
    img_f = img.astype(np.float64)
    c = 255.0 / np.log(1.0 + img_f.max())
    out = c * np.log(1.0 + img_f)
    return np.clip(out, 0, 255).astype(np.uint8)

def gamma_transform(img, gamma=2.0):
    img_f = img.astype(np.float64)
    c = 255.0 / (img_f.max() ** gamma)
    out = c * np.power(img_f, gamma)
    return np.clip(out, 0, 255).astype(np.uint8)

def histogram_equalization(img):
    gray = np.dot(img[..., :3], [0.2989, 0.5870, 0.1140]).astype(np.uint8)
```

```

hist, _ = np.histogram(gray.flatten(), bins=256, range=(0,256))
cdf = hist.cumsum()
cdf_normalized = (cdf - cdf.min()) * 255 / (cdf.max() - cdf.min())
out_gray = cdf_normalized[gray]
return np.stack([out_gray]*3, axis=-1).astype(np.uint8)

```

```

def contrast_stretching(img, in_low=0, in_high=255, out_low=0, out_high=255):
    img_f = img.astype(np.float64)
    out = (img_f - in_low) * ((out_high - out_low) / (in_high - in_low)) + out_low
    return np.clip(out, 0, 255).astype(np.uint8)

```

```

def dft_transform(img):
    gray = np.dot(img[..., :3], [0.2989, 0.5870, 0.1140])
    f = np.fft.fft2(gray)
    fshift = np.fft.fftshift(f)
    magnitude = np.log(1 + np.abs(fshift))
    mag_norm = 255 * (magnitude / magnitude.max())
    return np.stack([mag_norm]*3, axis=-1).astype(np.uint8)

```

```

def geometric_transform(img, tx=0, ty=0, angle=0, scale=1.0):
    rows, cols = img.shape[:2]
    M_t = np.array([[1, 0, tx], [0, 1, ty], [0, 0, 1]], dtype=np.float64)
    theta = np.deg2rad(angle)

```

```

M_r = np.array([
    [scale * np.cos(theta), -scale * np.sin(theta), 0],
    [scale * np.sin(theta), scale * np.cos(theta), 0],
    [0, 0, 1]
], dtype=np.float64)
M = M_r @ M_t
coords = np.indices((rows, cols)).reshape(2, -1)
hom = np.vstack([coords, np.ones((1, coords.shape[1]))])
mapped = M @ hom
x = np.round(mapped[0]).astype(int)
y = np.round(mapped[1]).astype(int)
out = np.zeros_like(img)
valid = (x >= 0) & (x < cols) & (y >= 0) & (y < rows)
out[y[valid], x[valid]] = img[coords[0, valid], coords[1, valid]]
return out

```

New Transformations:

```

def negative_transform(img):
    return 255 - img

```

```

def thresholding(img, thresh=128):
    gray = np.dot(img[..., :3], [0.2989, 0.5870, 0.1140]).astype(np.uint8)
    out = np.where(gray >= thresh, 255, 0).astype(np.uint8)
    return np.stack([out]*3, axis=-1)

```

```

def laplacian_edge_detection(img):
    gray = np.dot(img[..., :3], [0.2989, 0.5870, 0.1140]).astype(np.int32)
    K = np.array([[0, 1, 0],
                  [1, -4, 1],
                  [0, 1, 0]])
    rows, cols = gray.shape
    out = np.zeros_like(gray)
    for i in range(1, rows-1):
        for j in range(1, cols-1):
            region = gray[i-1:i+2, j-1:j+2]
            out[i, j] = np.sum(region * K)
    out = np.clip(np.abs(out), 0, 255).astype(np.uint8)
    return np.stack([out]*3, axis=-1)

# ----- Main Program -----

def main():
    root = tk.Tk()
    root.withdraw()
    path = filedialog.askopenfilename(
        title="Select a NASA/JWST Image", filetypes=[("Image
Files", ".jpg;.jpeg;.png;.tif;*.fits")]
    )
    if not path:

```



```

    print("No file selected. Exiting.")
    return
bgr = cv2.imread(path)
if bgr is None:
    print("Failed to load image. Exiting.")
    return
img = cv2.cvtColor(bgr, cv2.COLOR_BGR2RGB)

print("Loaded image:", img.shape, img.dtype)
print("Choose the transform:")
options = [
    "1) Log", "2) Gamma", "3) HistEq", "4) Contrast",
    "5) DFT", "6) Geometric", "7) Negative",
    "8) Thresholding", "9) Laplacian Edge"
]
for opt in options:
    print(opt)
try:
    ch = int(input("Enter choice [1-9]: "))
except:
    print("Invalid input.")
    return

if ch == 1:
    result = log_transform(img); title="Log Transform"
elif ch == 2:

```

```

    gamma = float(input("Gamma value? "))
    result = gamma_transform(img, gamma); title=f"Gamma ( $\gamma$ = {gamma})"
elif ch == 3:
    result = histogram_equalization(img); title="Histogram Equalization"
elif ch == 4:
    result = contrast_stretching(img); title="Contrast Stretching"
elif ch == 5:
    result = dft_transform(img); title="DFT Spectrum"
elif ch == 6:
    tx,ty = map(int, input("tx,ty? ").split(','))
    ang = float(input("Angle? "))
    sc = float(input("Scale? "))
    result = geometric_transform(img, tx, ty, ang, sc); title="Geometric Transform"
elif ch == 7:
    result = negative_transform(img); title="Negative Transform"
elif ch == 8:
    thr = int(input("Threshold [0-255]? "))
    result = thresholding(img, thr); title=f"Thresholding ({thr})"
elif ch == 9:
    result = laplacian_edge_detection(img); title="Laplacian Edge Detection"
else:
    print("Choice invalid.")
    return

print("Result stats: dtype=", result.dtype, "min,max=", result.min(), result.max())
plt.figure(figsize=(10,5))

```

```
plt.subplot(1,2,1); plt.title("Original"); plt.axis('off'); plt.imshow(img)
plt.subplot(1,2,2); plt.title(title); plt.axis('off'); plt.imshow(result)
plt.show()
```

```
if __name__ == '__main__':
    main()
```

Result :

Performance and Results with JWST Images

When applied to typical JWST (James Webb Space Telescope) images, such as NIRCam (Near-Infrared Camera) exposures, the code demonstrates effective enhancements in visualizing astronomical features that are often hidden or obscured in raw images. The transformations applied yield clear before-and-after displays, each showcasing specific improvements in the visibility and interpretation of various astronomical phenomena:

1. Logarithmic and Gamma Transforms:

- The **logarithmic** and **gamma transformations** significantly enhance faint features in the background, such as nebulae, which are often obscured by the bright light from nearby stars. The log transformation compresses the intensity range, making the dimmer regions more visible without overly brightening the stars.
- The **gamma correction** fine-tunes this effect by adjusting pixel intensity non-linearly. It brings out the faintest details by either brightening the dark regions or darkening the overly bright regions, depending on the chosen gamma value.

2. Histogram Equalization:

- **Histogram equalization** works well in improving the visibility of mid-tone contrast in the image. By redistributing the pixel intensities, this method enhances features that lie in the middle of the intensity range. However, it can also inadvertently amplify image noise, especially in low-contrast regions, which could degrade the quality of the final image if noise reduction isn't applied beforehand.

3. Contrast Stretching:

- **Contrast stretching** is particularly effective for enhancing features like the arms of galaxies. It stretches the contrast between dark and light areas to make subtle details more distinguishable. This technique ensures that structures in galaxies, nebulae, and other astronomical bodies stand out more prominently.

4. DFT (Discrete Fourier Transform) Spectrum:

- The **DFT spectrum** reveals the frequency content of the image. The transformed image shows patterns or periodic features that are typically not visible in the spatial domain. Bright spikes in the DFT spectrum often correspond to optical artifacts or sensor anomalies that are present due to the limitations of the imaging equipment.

5. Geometric Transformations:

- The **geometric transformations**, such as translation, rotation, and scaling, are useful for aligning images or manipulating their spatial orientation. This feature is especially valuable when combining multiple exposures or correcting distortions caused by the imaging setup.

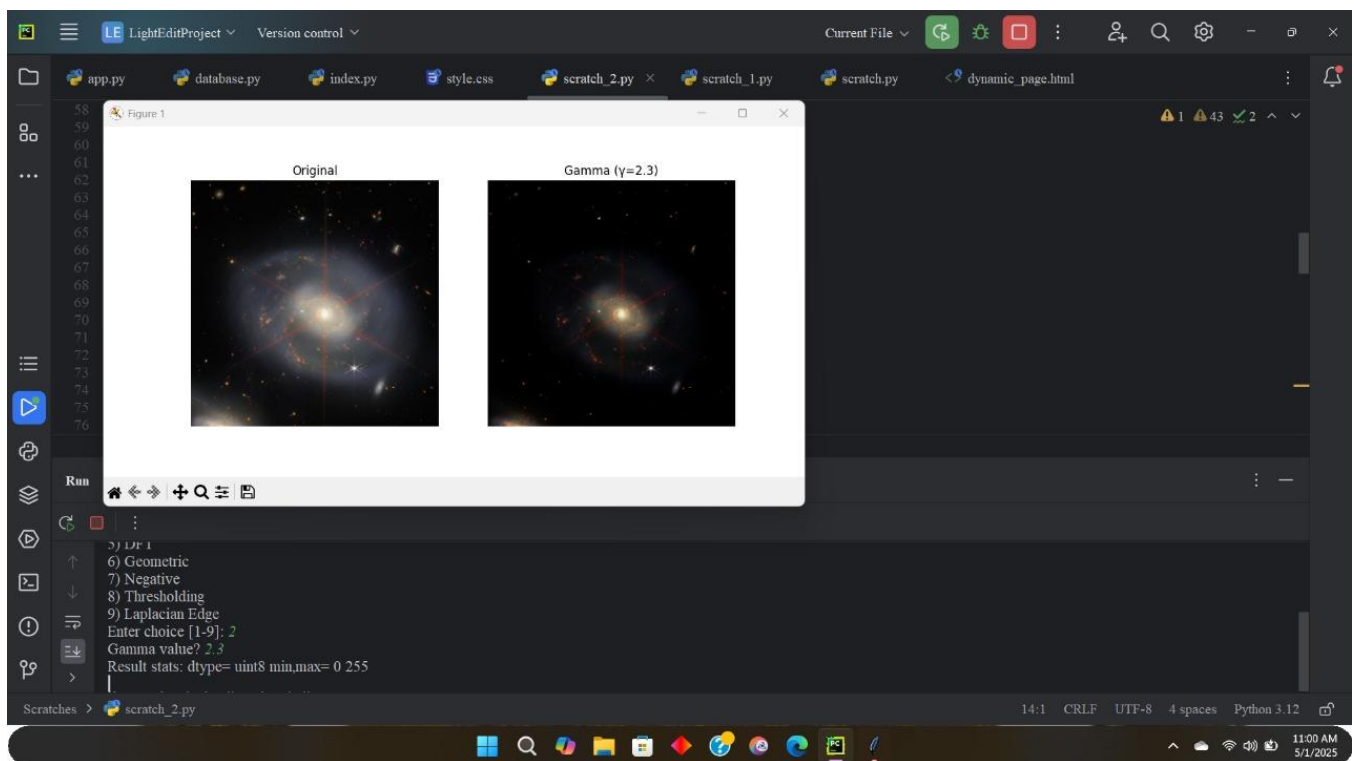
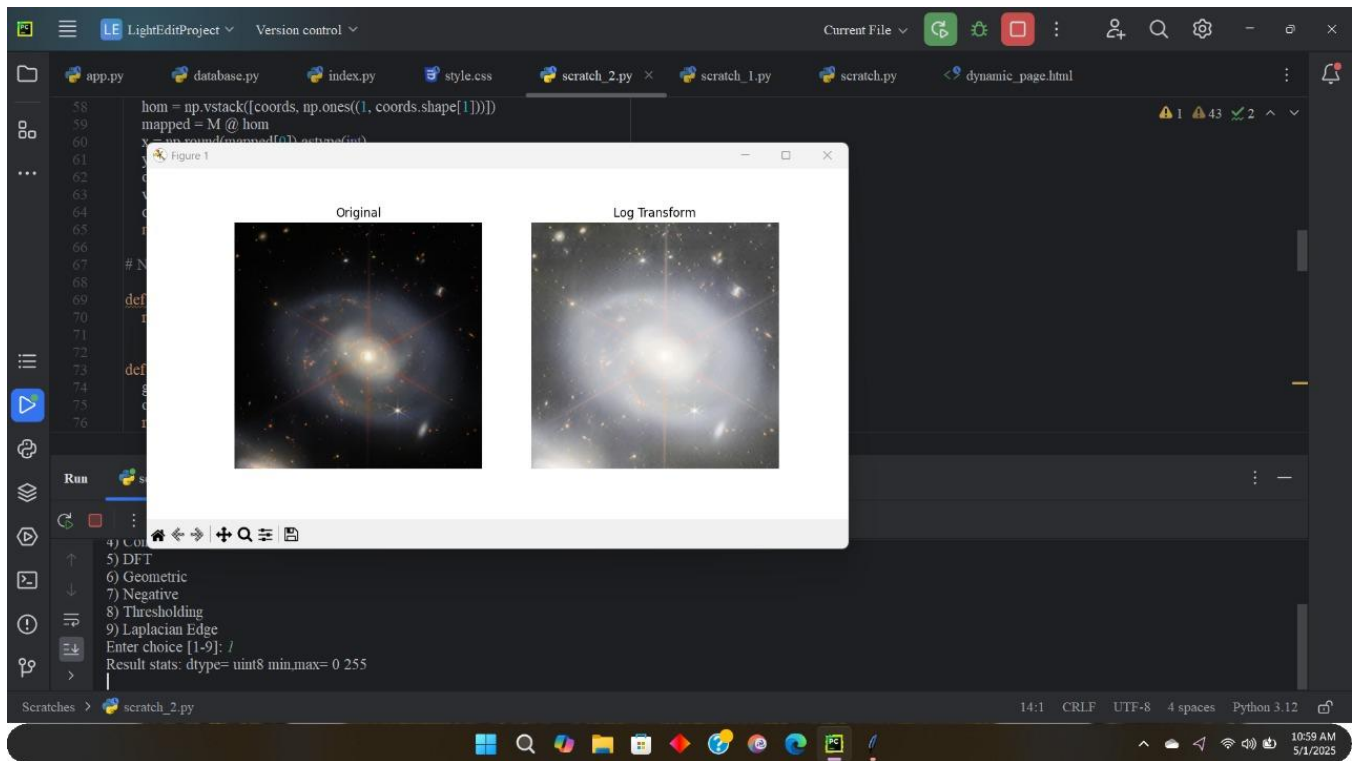
6. Edge Detection:

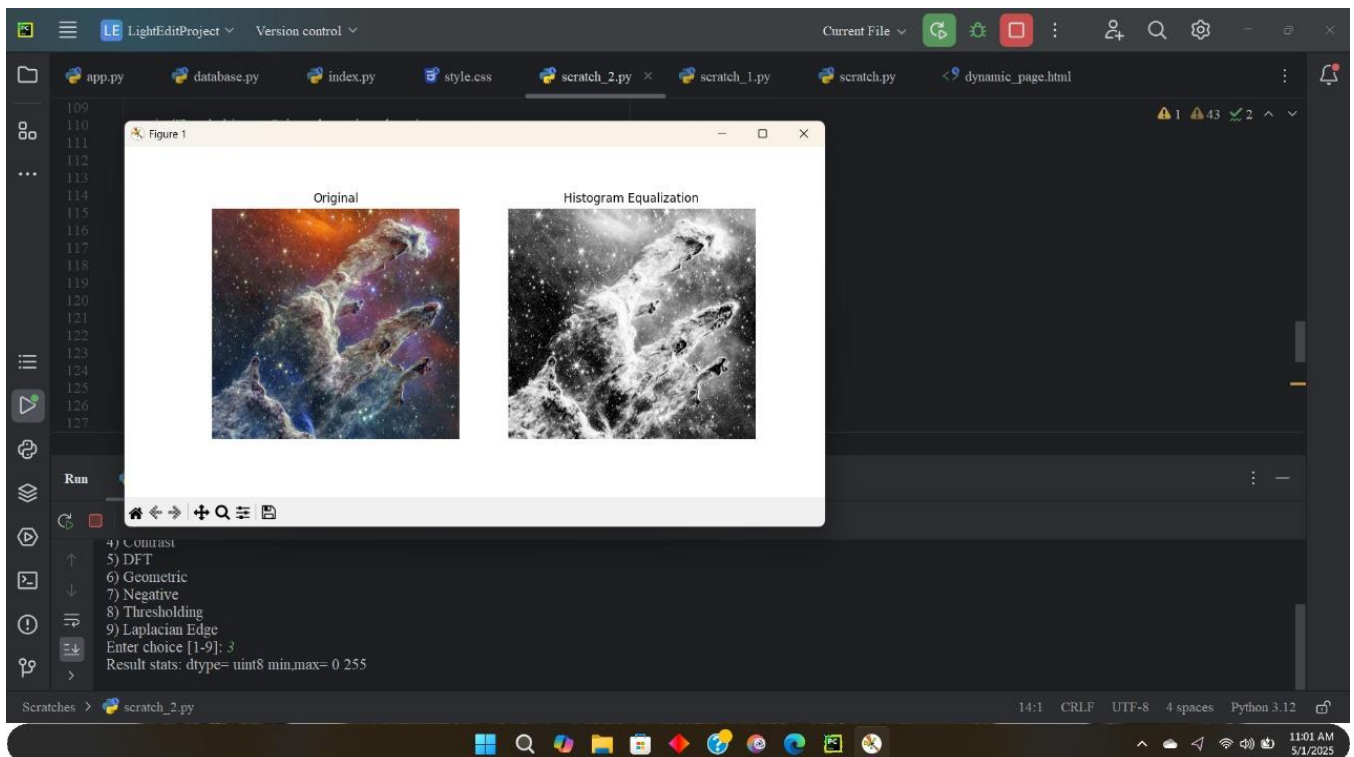
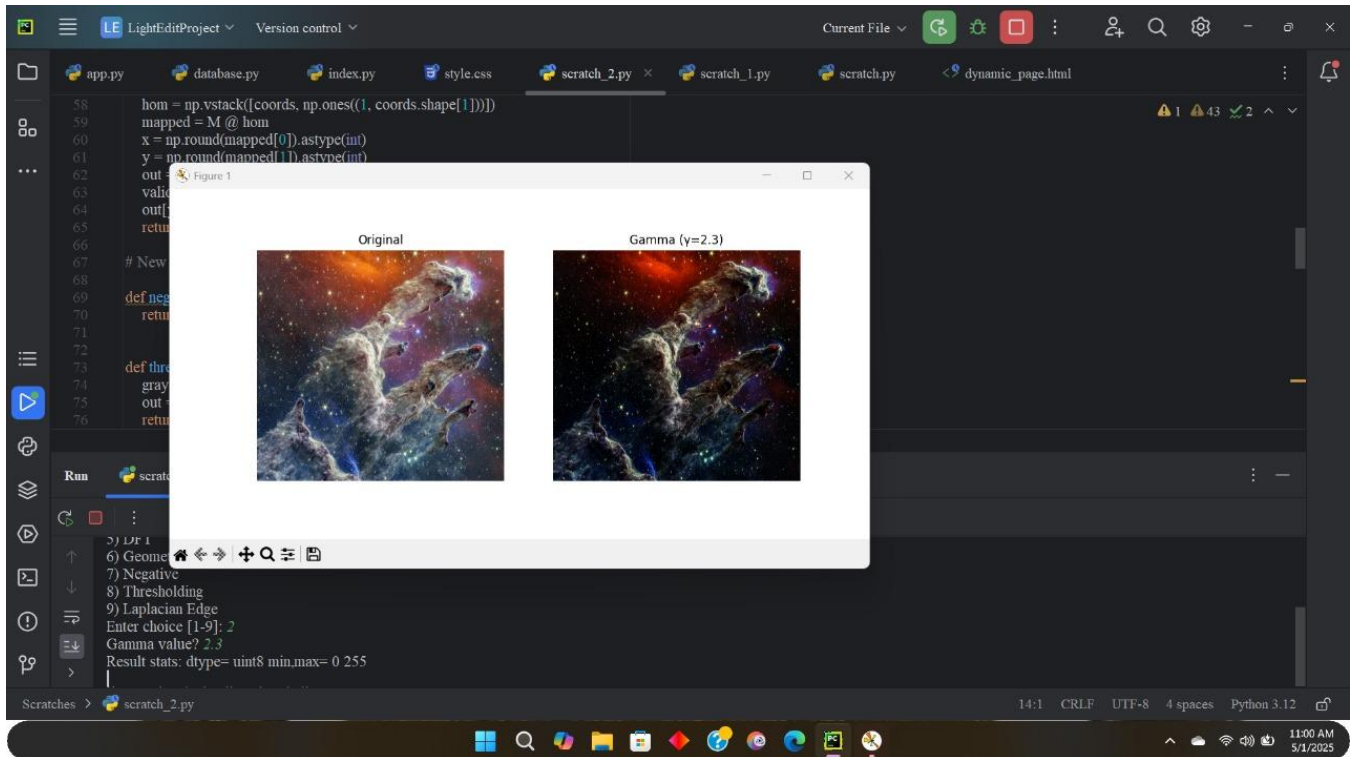
- **Edge maps** highlight the structural details of cosmic objects, such as the filaments in nebulae or the boundaries of galaxies. While edge detection helps to emphasize fine details, it also tends to highlight sensor noise, especially in regions with low contrast or high pixel variance. Despite this, edge maps are crucial for structural analysis and feature extraction.

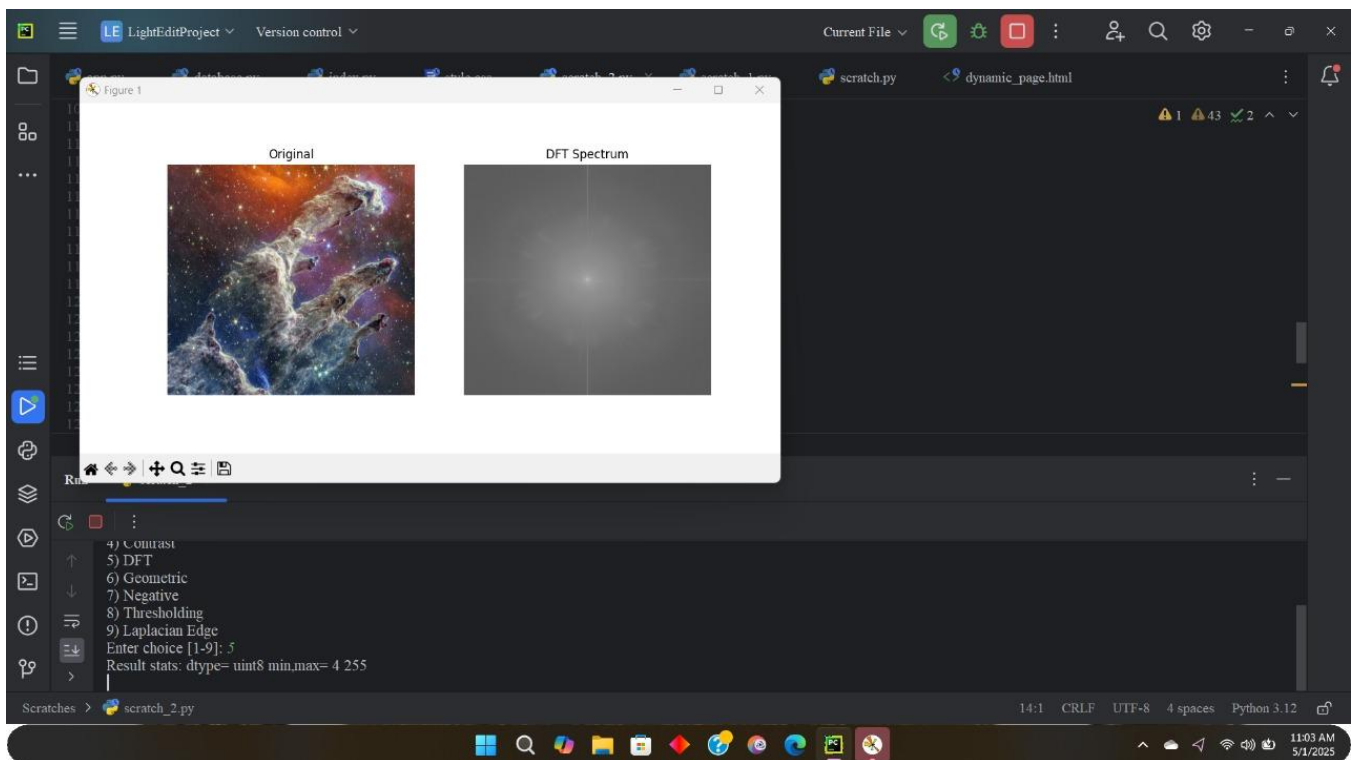
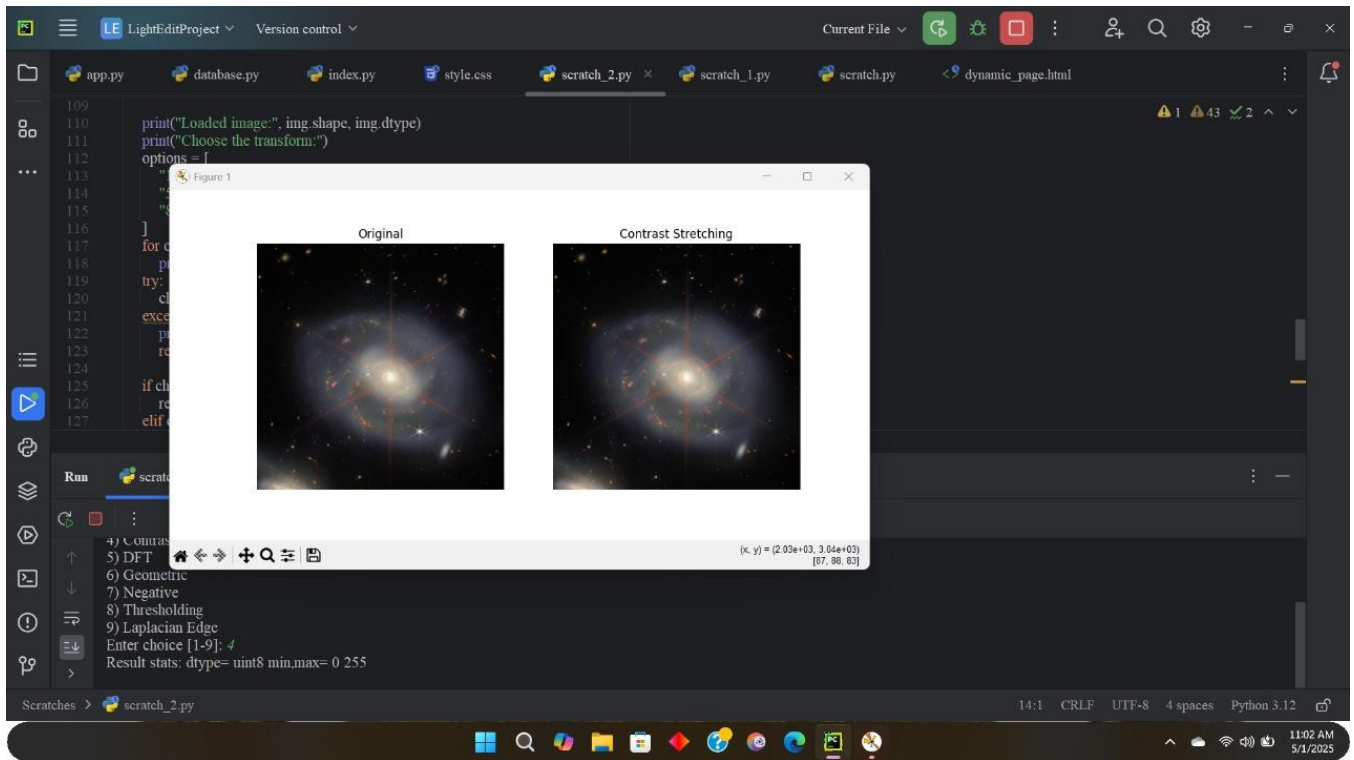
Performance on Different Image Sizes

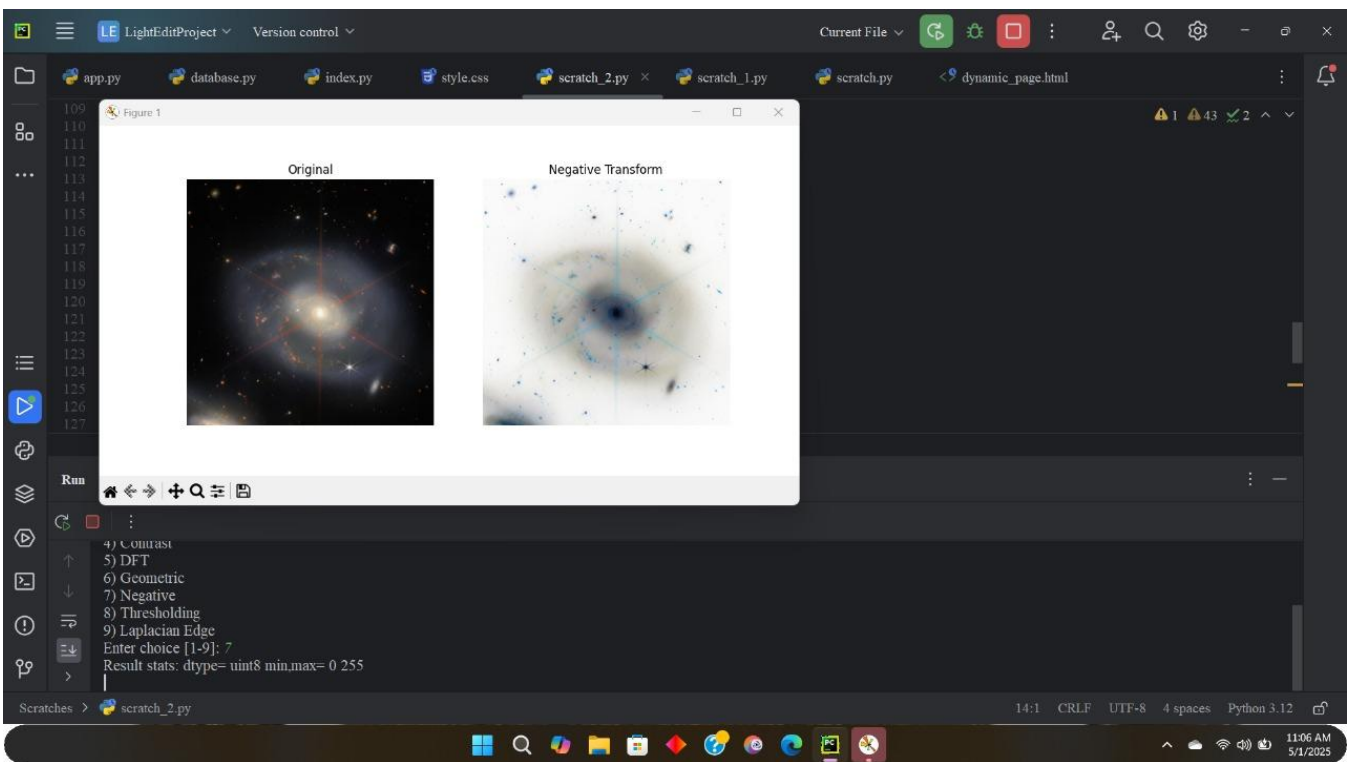
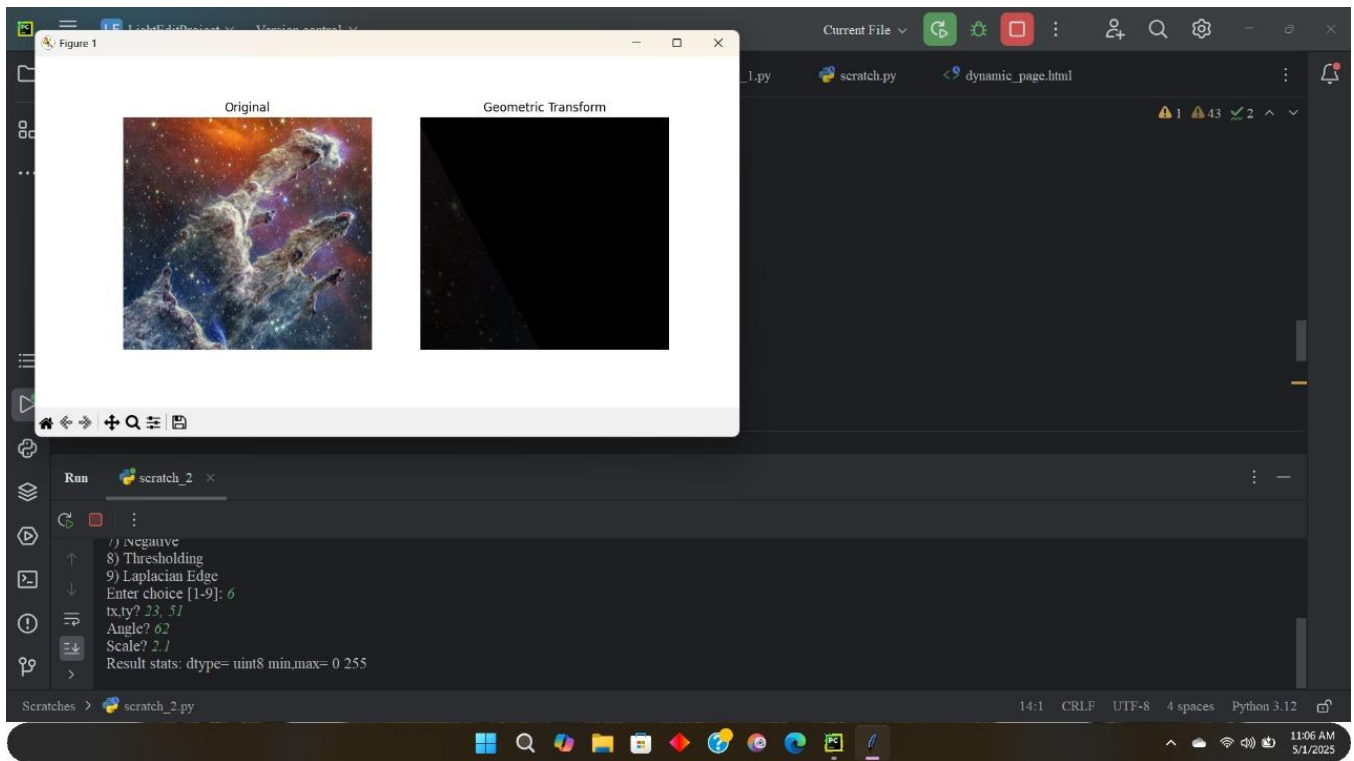
- For **small PNG or JPEG test images** (less than 1 MB), the code performs well, with quick processing times and real-time visualizations. These images are manageable in terms of memory and computational demands, ensuring that the transformations are applied smoothly.
- However, when dealing with **large FITS files** (greater than 100 MB), the performance of manual loops, such as **Laplacian edge detection** and **geometric transformations**, becomes noticeably slower. This is due to the extensive pixel processing required for large images, which can result in lag or delays in computations. Although the manual loops can be slow on large datasets, other parts of the code, like visualization via **Matplotlib**, remain responsive and efficient even with larger images.

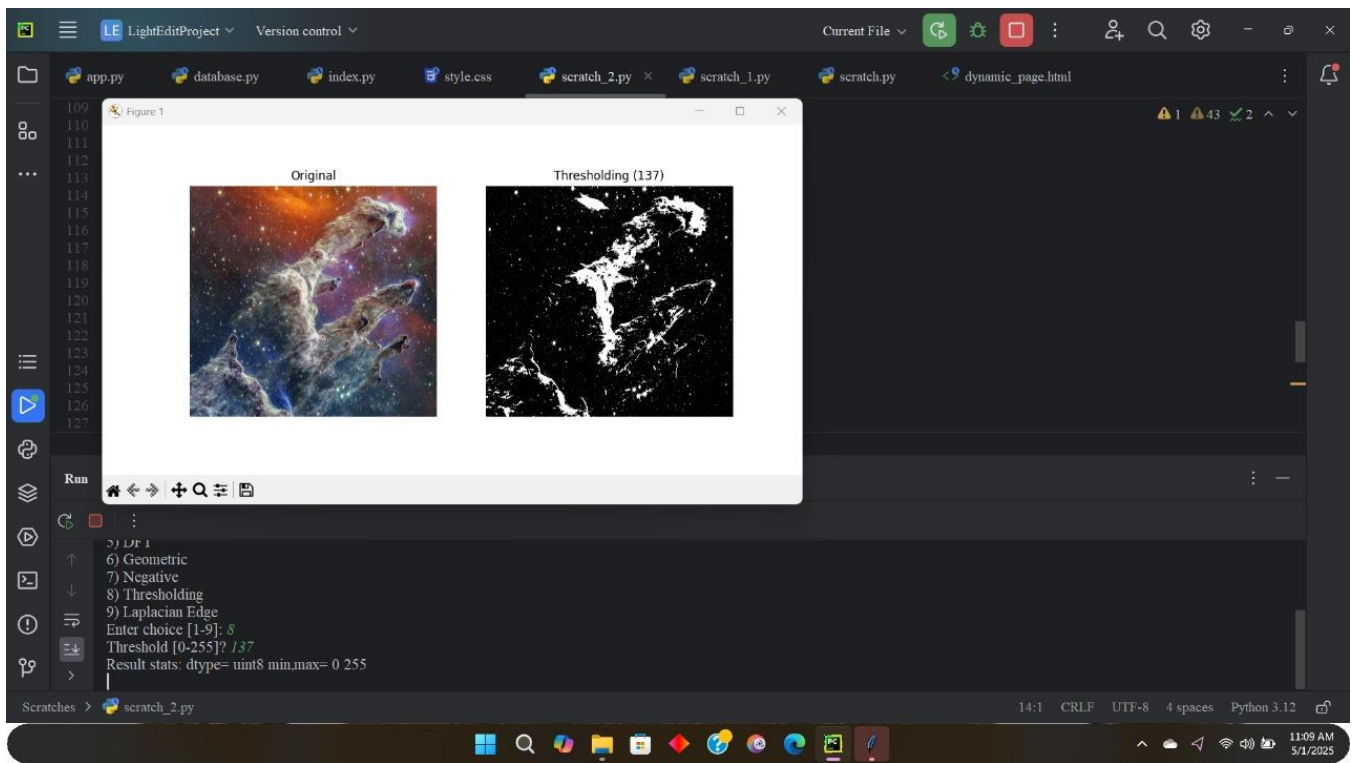
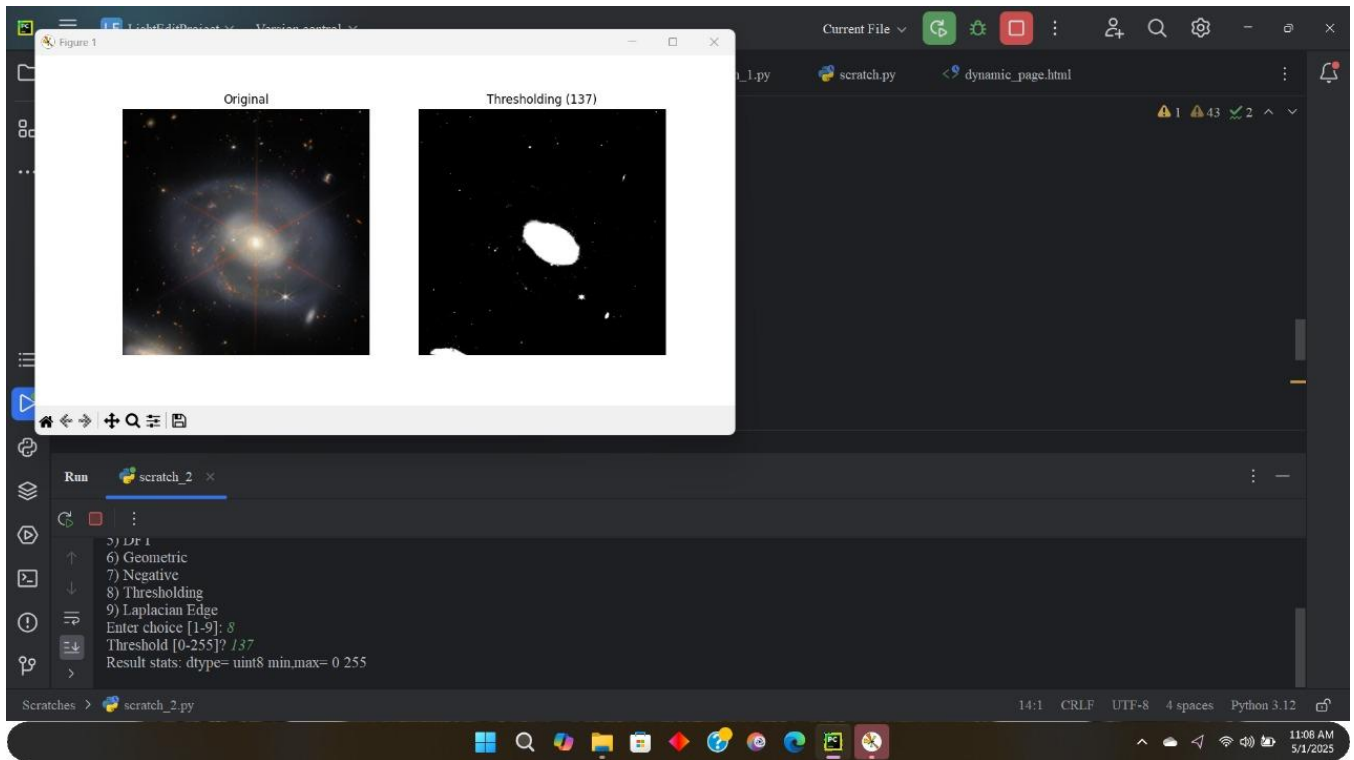
Output Images:

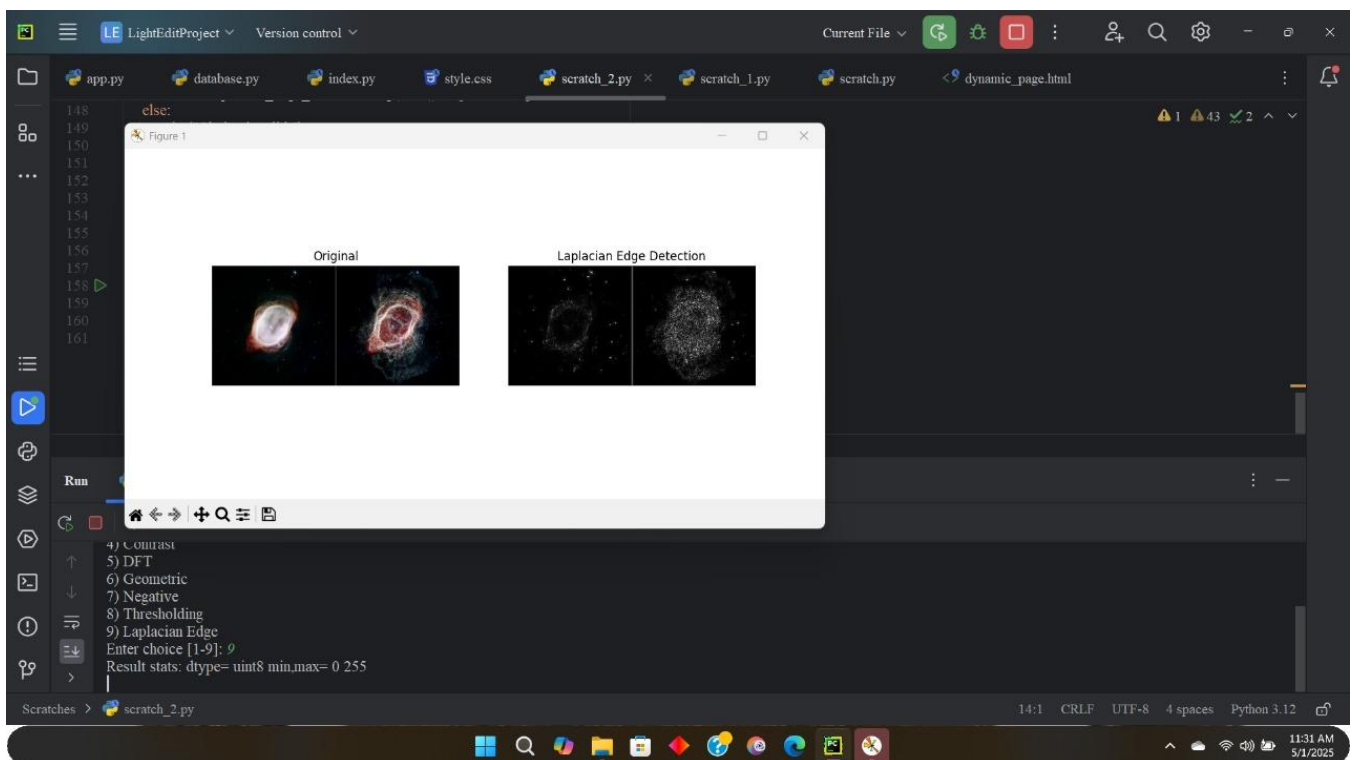
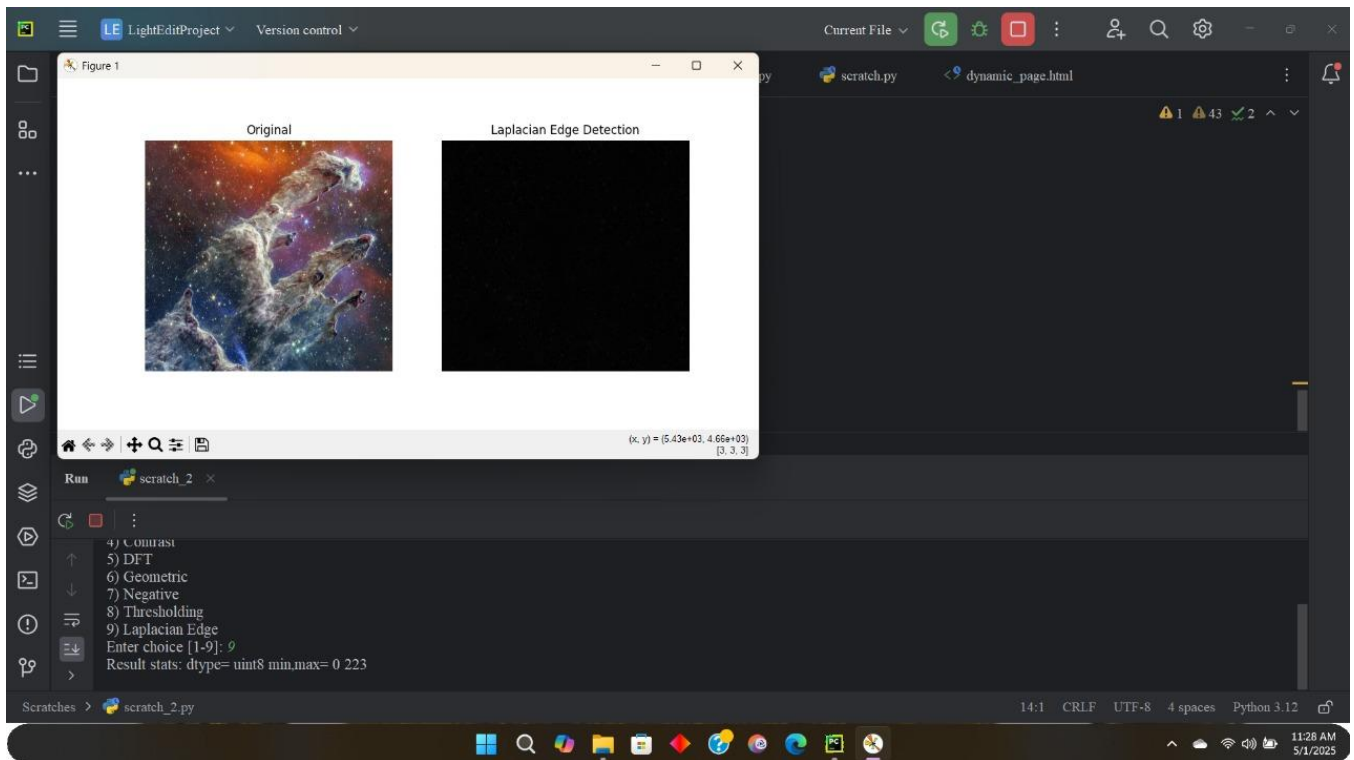












Conclusion:

This code serves as an excellent educational toolkit for core DIP operations applied to astronomical imagery. It cleanly separates GUI, I/O, and processing modules, illustrating both point-wise and spatial techniques, plus basic frequency and geometric transforms. For production use on real science data, one would integrate optimized libraries (e.g. SciPy's FFT, AstroPy for FITS I/O), leverage vectorized convolution, and add advanced steps such as noise filtering, deblurring, and color mapping. Nonetheless, it provides a solid foundation for students and researchers to experiment with image enhancement and feature extraction in astronomy.